CHAPTER 2

Discrete Compound Interest & Multi-Period Accumulation

Geometric Sequences, Compounding Frequencies, Monotonicity Theorems, and Logarithmic Doubling Time

---

2.1 Definition & Mathematical Axioms

Compound Interest is the engine of capital accumulation in modern finance. Unlike linear simple interest, compound accumulation operates on the principle that **accrued interest is periodically added to the principal**, generating additional return in all subsequent periods (interest on interest).

Definition 2.1 (Discrete Compounding Accumulation)

Let initial capital be $P > 0$, nominal annual interest rate be $r \geq 0$ (decimal fraction: $r = R / 100$), compounding frequency be $n \geq 1$ (sub-intervals per year: annual n=1, semi-annual n=2, quarterly n=4, monthly n=12), and total time horizon be $t \geq 0$ years.
The **periodic interest rate** is $i = r / n$, and total compounding periods is $N = n \cdot t$.
The **terminal accumulated value** $A(t)$ and **net compound interest** $I_{\text{comp}}(t)$ are:

$$A(t) = P \cdot (1 + r / n)^{n \cdot t}$$

$$I_{\text{comp}}(t) = A(t) - P = P\,[(1 + r / n)^{n \cdot t} - 1]$$

The accumulation factor is $a(t) = (1 + r / n)^{nt}$, satisfying $a(0) = 1$ and $a'(t) > 0$.

2.2 First-Principles Derivation & Frequency Monotonicity

Geometric Progression Derivation:
Let A_k denote wealth at the end of period k with periodic rate $i = r/n$:

- Period 1: $A_1 = P + P{\cdot}i = P(1 + i)$
- Period 2: $A_2 = A_1(1 + i) = P(1 + i)^2$
- By induction for $N = nt$ periods: $A_N = P(1 + i)^N = P(1 + r/n)^{nt}$. ■

Theorem 2.1 (Strict Monotonicity with Respect to Compounding Frequency)

For any fixed annual rate $r > 0$ and time horizon $t > 0$, the accumulation factor $a(n) = (1 + r/n)^{nt}$ is a **strictly increasing function** of compounding frequency n:

$$n_1 < n_2 \Rightarrow (1 + r/n_1)^{n_1 t} < (1 + r/n_2)^{n_2 t} < e^{rt}$$

Proof: Let $x = r / n > 0$, so $n = r / x$. Taking the natural logarithm: $\ln a(x) = (rt / x) \ln(1 + x) = rt \cdot [\ln(1 + x) / x]$. Differentiating with respect to x: $d/dx\ [\ln(1 + x) / x] = [x/(1 + x) - \ln(1 + x)] / x^2$.
From real analysis, $\ln(1 + x) > x / (1 + x)$ for all $x > 0$. Thus the derivative with respect to x is strictly negative. Since $x = r / n$ decreases as n increases, $[\ln(1 + x) / x]$ strictly increases with n. Applying the strictly increasing exponential function confirms $a(n)$ is strictly increasing in n. ■

2.3 Step-by-Step Worked Examples

Below are two formal case studies solved by hand: multi-frequency corporate accumulation and exact logarithmic doubling time.

Example 2.1: Multi-Frequency Corporate Fixed Deposit

Problem Statement: An investor commits principal ***P* = INR 500,000** at nominal interest rate ***R* = 8.40% per annum** for ***t* = 5 years**. Compute terminal accumulated value under:

- **Case A:** Annual Compounding ($n = 1$)
- **Case B:** Quarterly Compounding ($n = 4$)
- **Case C:** Monthly Compounding ($n = 12$)

Step-by-Step Analytical Solution:

Case A (Annual Compounding, *n* = 1):

- Periodic rate: $i = 0.084 / 1 = 0.084$
- Total periods: $N = 1 \times 5 = 5$ periods
- Calculation: $A_{annual} = 500{,}000 \times (1.084)^5 = 500{,}000 \times 1.4968593 =$ **INR 748,429.65**

Case B (Quarterly Compounding, *n* = 4):

- Periodic rate: $i = 0.084 / 4 = 0.021$
- Total periods: $N = 4 \times 5 = 20$ periods
- Calculation: $A_{quarterly} = 500{,}000 \times (1.021)^{20} = 500{,}000 \times 1.5153830 =$ **INR 757,691.50**

Case C (Monthly Compounding, *n* = 12):

- Periodic rate: $i = 0.084 / 12 = 0.007$
- Total periods: $N = 12 \times 5 = 60$ periods
- Calculation: $A_{monthly} = 500{,}000 \times (1.007)^{60} = 500{,}000 \times 1.5197581 =$ **INR 759,879.05**

Frequency Gain Analysis:

- Monthly vs Annual Surplus: 759,879.05 – 748,429.65 = **INR 11,449.40** excess wealth.
- Quarterly vs Annual Surplus: 757,691.50 – 748,429.65 = **INR 9,261.85** excess wealth.
- *Takeaway:* Increasing the compounding frequency from annual to monthly produces an incremental gain of 2.29% on initial principal with zero additional market risk.

Example 2.2: Exact Doubling Time vs. The Heuristic 'Rule of 72'

Problem Statement: Determine the exact time τ_2 required to double capital at nominal rate ***R* = 9.00%** compounded quarterly ($n = 4$). Compare against the popular 'Rule of 72'.

Step-by-Step Analytical Solution:

Step 1 (Derivation of Exact Doubling Time Equation):

Set terminal value equal to twice principal: $A(\tau_2) = 2P$

$2P = P\,(1 + r/n)^{n \cdot \tau_2} \;\Rightarrow\; 2 = (1 + r/n)^{n \cdot \tau_2}$

Applying the natural logarithm to both sides:

$\ln(2) = n \cdot \tau_2 \cdot \ln(1 + r/n)$

Solving algebraically for duration τ_2:

$$\boldsymbol{\tau_2 = \ln(2) / [n \cdot \ln(1 + r/n)]}$$

Step 2 (Numerical Evaluation):

- Parameters: $r = 0.09$, $n = 4 \;\Rightarrow\; i = 0.09 / 4 = 0.0225$
- Logarithmic values: $\ln(2) \approx 0.69314718$, $\ln(1.0225) \approx 0.02225059$
- Denominator: $4 \times 0.02225059 = 0.08900236$
- Exact Doubling Horizon: $\tau_2 = 0.69314718 / 0.08900236 =$ **7.78796 years** (≈ 7 yrs, 9.5 mos).

Step 3 (Evaluation of Rule of 72 Heuristic):

$\tau_{heuristic} = 72 / R = 72 / 9 =$ **8.0000 years**.

Step 4 (Error & Analytical Insight):

- Absolute error: $\tau_{heuristic} - \tau_2 = 8.0000 - 7.7880 =$ **+0.212 years** (+2.54 months).
- Relative error: $0.212 / 7.788 \approx$ **+2.72%** overestimation.
- *Theoretical origin:* The Rule of 72 stems from $\ln(2) \approx 0.693 \approx 0.72$ (adjusted to account for compounding drift at mid-range interest rates). In algorithmic financial systems, the closed-form logarithmic formula is required to avoid cumulative timing errors.

2.4 Algorithmic Implementation in Python

Below is the production-grade Python implementation from models/time_value.py, featuring frequency validation, type safety, and doubling time analysis:

Listing: models/time_value.py — Discrete Compounding Engine

```
import math

def compound_interest_amount(
    principal: float, annual_rate: float, time_years: float, compounding_frequency: int = 1
) -> float:
    """
    Computes terminal wealth under discrete compounding: A = P * (1 + r / n) ** (n * t).
    """
    if principal < 0.0 or annual_rate < 0.0 or time_years < 0.0 or compounding_frequency <= 0:
        raise ValueError("Invalid parameters: financial arguments must be non-negative, n >= 1.")
    periodic_rate = annual_rate / compounding_frequency
    total_periods = compounding_frequency * time_years
    return float(principal * ((1.0 + periodic_rate) ** total_periods))


def compound_doubling_time(annual_rate: float, compounding_frequency: int = 1) -> float:
    """
    Computes exact doubling time: tau_2 = ln(2) / [n * ln(1 + r / n)].
    """
    if annual_rate <= 0.0 or compounding_frequency <= 0:
        raise ValueError("Rate must be strictly positive and n >= 1 for doubling.")
    periodic_rate = annual_rate / compounding_frequency
    return float(math.log(2.0) / (compounding_frequency * math.log(1.0 + periodic_rate)))
```

2.5 Mathematical Properties Summary

Mathematical Property	Specification & Formula	Analytical Significance
Domain & Constraints	$P > 0$, $r \geq 0$, $t \geq 0$, $n \geq 1$	Discrete periodic compounding structure
Accumulation Factor	$a(t) = (1 + r/n)^{nt}$	Superlinear exponential growth operator
Frequency Limit	$\lim_{n \to \infty} a(t) = e^{rt}$	Upper supremum bound achieved under continuous compounding
Convexity	$d^2a/dt^2 > 0 \ \ \forall\ t \geq 0$	Strict convexity drives compounding velocity acceleration
Doubling Time	$\tau_2 = \ln(2) / [n \ln(1 + r/n)]$	Exact logarithmic capital replication milestone